

I am learning k-Nearest Neighbors (KNN) I want you to write simplest python script so that I can implement and understand k-Nearest Neighbors Classification and regression . Use some online dataset that is good for logistic regression.

KNN Classification Example

This script uses the Breast Cancer Wisconsin dataset from scikit-learn, which is a binary classification problem (malignant vs. benign tumors). It's suitable for classification tasks like those often done with logistic regression, but here we'll use KNN. The script loads the data, trains a KNN classifier with $k=5$, and evaluates it.

Python

```
# Import necessary libraries
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# Load the dataset
data = load_breast_cancer()
X = data.data # Features
y = data.target # Labels (0: malignant, 1: benign)

# Split data into training and testing sets (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,

# Create KNN classifier with k=5
knn_clf = KNeighborsClassifier(n_neighbors=5)

# Train the model
knn_clf.fit(X_train, y_train)

# Make predictions on the test set
y_pred = knn_clf.predict(X_test)

# Evaluate accuracy
```

```
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.2f}")
```

To understand:

- KNN classification works by finding the k nearest points to a new sample in the feature space and assigning the majority class among them.
- Run this script; it should print an accuracy around 0.96 (may vary slightly).
- Experiment by changing `n_neighbors` to see how k affects performance.

KNN Regression Example

This script uses the Diabetes dataset from scikit-learn, which predicts a continuous disease progression score based on features like age, BMI, etc. It's a regression task.

Python

```
# Import necessary libraries
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsRegressor
from sklearn.metrics import mean_squared_error

# Load the dataset
data = load_diabetes()
X = data.data # Features
y = data.target # Target (disease progression)

# Split data into training and testing sets (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,

# Create KNN regressor with k=5
knn_reg = KNeighborsRegressor(n_neighbors=5)

# Train the model
knn_reg.fit(X_train, y_train)

# Make predictions on the test set
y_pred = knn_reg.predict(X_test)

# Evaluate with Mean Squared Error (MSE)
```

```
mse = mean_squared_error(y_test, y_pred)
print(f"Mean Squared Error: {mse:.2f}")
```

To understand:

- KNN regression works similarly but averages the target values of the k nearest points instead of voting on classes.
- Run this script; it should print an MSE around 3000–4000 (may vary).
- Again, try different `n_neighbors` values.

These scripts are self-contained and use built-in scikit-learn datasets—no need to download files. Install scikit-learn if needed (`pip install scikit-learn`). For deeper understanding, add code to visualize data or tune hyperparameters.

↳ Visualize decision boundaries

↳ KNN hyperparameter tuning

↳ Add confusion matrix